

# Guión de presentación

## TP3 — Perceptrones

Sistemas de Inteligencia Artificial · ITBA · 1Q 2026

| Persona | Bloque                                                        | Slides | Tiempo |
|---------|---------------------------------------------------------------|--------|--------|
| P1      | Introducción + Ej0 (validación) + Engine <a href="#">mlp/</a> | 1–6    | 5 min  |
| P2      | Ej1 (knowledge distillation) — <i>Tomás</i>                   | 7–12   | 5 min  |
| P3      | Ej2 (clasificación de dígitos)                                | 13–19  | 5 min  |
| P4      | Ej3 (Pack C, búsqueda del 98 %) + conclusiones                | 20–26  | 5 min  |

**Total: 20 minutos.** Las slides están en docs/slides\_presentacion.pdf. Companion técnico: docs/informe\_tp3.pdf. Documentación viva: docs/guia\_compañeros.md.

Convenciones: [\[Slide N\]](#) = referencia a slide. (0:30) = tiempo aproximado dentro del bloque. *en cursiva rojo* = referencia explícita a la consigna o al apunte. [texto en azul](#) = identificador del repo.

## Índice

|                                                               |          |
|---------------------------------------------------------------|----------|
| <b>1. Persona 1 — Introducción, Ej0 y Engine</b>              | <b>1</b> |
| 1.1. Apertura (0:00 – 0:40)                                   | 1        |
| 1.2. Filosofía: engine único (0:40 – 1:40)                    | 1        |
| 1.3. Ejercicio 0 — validación (1:40 – 3:30)                   | 2        |
| <b>2. Persona 2 — Ejercicio 1 (Knowledge Distillation)</b>    | <b>2</b> |
| 2.1. Setup del problema (0:00 – 1:00)                         | 2        |
| 2.2. Lineal vs No-Lineal (1:00 – 2:30)                        | 3        |
| 2.3. Threshold sweep (2:30 – 4:00)                            | 3        |
| <b>3. Persona 3 — Ejercicio 2 (Clasificación de dígitos)</b>  | <b>4</b> |
| 3.1. Datos y problema (0:00 – 0:40)                           | 4        |
| 3.2. Workflow Fase 1 + Fase 2 (0:40 – 2:00)                   | 4        |
| 3.3. Fase 2 valida la elección (2:00 – 3:30)                  | 4        |
| 3.4. Final eval — el descubrimiento (3:30 – 5:00)             | 5        |
| <b>4. Persona 4 — Ejercicio 3 (Pack C, búsqueda del 98 %)</b> | <b>5</b> |
| 4.1. Hipótesis y plan (0:00 – 0:40)                           | 5        |
| 4.2. El primer movimiento dominó (0:40 – 1:40)                | 5        |
| 4.3. Iteración Pack C (1:40 – 3:00)                           | 5        |
| 4.4. Análisis: qué movió la aguja (3:00 – 4:00)               | 6        |
| 4.5. Por qué no llegamos al 98 % (4:00 – 4:30)                | 6        |
| 4.6. Conclusiones del TP (4:30 – 5:00)                        | 6        |
| <b>Apéndice — preguntas anticipadas</b>                       | <b>7</b> |

## 1. Persona 1 — Introducción, Ej0 y Engine

**Objetivo de este bloque:** situar al jurado, explicar la filosofía de implementación, validar que el engine funciona (XOR, AND), y entregar limpio a Persona 2.

### 1.1 Apertura (0:00 – 0:40)

[Slide 1, portada] “Buenas. Trabajo Práctico 3, perceptrones. Somos cuatro y nos vamos a turnar en orden de los ejercicios. Yo arranco con la introducción y el ejercicio 0 de validación. Al final esperamos preguntas.”

[Slide 2, qué pide el TP] “El enunciado pide construir perceptrones desde cero, en NumPy puro — *sin sklearn estimators, sin PyTorch, sin TensorFlow como capas*. Tres ejercicios principales más una validación. Ejercicio 1 es detección de fraude por knowledge distillation; ejercicio 2 es clasificación de dígitos manuscritos con un MLP; y ejercicio 3 es lo mismo que el 2 pero apuntando a un *accuracy mayor o igual a 98 % sobre el test set*.”

### 1.2 Filosofía: engine único (0:40 – 1:40)

[Slide 3, arquitectura del repo] “Tomamos una decisión arquitectural temprana: en vez de armar tres scripts independientes, construimos un **módulo mlp/ reutilizable** y resolvemos los ejercicios cambiando solo **configs JSON**. Pueden verlo en el repo: `mlp/network.py`, `mlp/optimizers.py`,

mlp/activations.py, mlp/losses.py. El ejercicio 1, que es lo que va a presentar el siguiente, vive aparte porque ya estaba implementado por Tomás antes de esta unificación.”

**Cue visual:** mostrar el árbol de directorios. Si te preguntan “¿por qué un solo engine?”, respuesta corta: “una pipeline, un set de tests (78 verde), una sola fuente de bugs.”

[Slide 4, anatomía del engine] “El engine tiene cuatro componentes intercambiables vía config: activaciones (sigmoid, tanh, relu, identity, softmax), losses (MSE, BCE, cross\_entropy), inicializadores (uniform, He, Xavier — y un *auto* que elige según la activación de cada capa), y optimizadores (SGD, Momentum, Adam). El bias usa el truco de prependear una columna de unos a la entrada de cada capa: así cada peso  $W^{(l)}$  tiene shape  $(n_{l+1}, n_l + 1)$  con la columna 0 como bias. Eso simplifica el backprop.”

### 1.3 Ejercicio 0 — validación (1:40 – 3:30)

[Slide 5, AND con step perceptron] “El ejercicio 0 valida que el código está bien antes de tirarlo a problemas grandes. Primero, perceptrón con función escalón sobre el AND lógico, en *representación bipolar como pide el apunte* — entradas en  $\{-1, +1\}$ , salida esperada  $[-1, -1, -1, +1]$ . El test pasa con error cero. Eso es ejercicio0/step\_perceptron.py.”

[Slide 6, XOR con MLP] “Lo más interesante del ejercicio 0 es el XOR — es el caso clásico que un perceptrón simple no puede resolver. Necesitás una capa oculta no lineal. Probamos dos arquitecturas: la mínima  $[2, 2, 1]$  y otra con capa extra  $[2, 3, 2, 1]$ . Los configs están en ejercicio0/configs/. Ambas convergen a train\_loss menor a 0.1 en pocas epochs.

El XOR no es ornamental: es la red de seguridad del engine. Si rompemos backprop, XOR deja de converger *antes* que dígitos. De hecho durante el desarrollo encontramos cuatro bugs reales gracias a estos tests: predicción bipolar mal codificada en `predict()`, identity activation que aliasaba su input, mean/std del summary que perdían filas por dict-spread overwrite, y `numerical_grad` que devolvía garbage para int dtypes.”

**Transición (4:30 – 5:00):** “Eso valida el engine para los ejercicios siguientes. Pero el ejercicio 1, knowledge distillation, fue implementado por Tomás antes de esta unificación con su propia pipeline. Le paso a [P2].”

## 2. Persona 2 — Ejercicio 1 (Knowledge Distillation)

**Objetivo:** explicar el setup de distillation, comparar lineal vs no-lineal, justificar el threshold óptimo. Si quien presenta NO es el autor, dejar claro al inicio: “este ejercicio lo implementé hace un mes con scripts independientes; el resto del TP lo unificamos después.”

### 2.1 Setup del problema (0:00 – 1:00)

[Slide 7, problema] “Knowledge distillation: *el enunciado nos da un BigModel pre-entrenado que clasifica fraude* — nosotros lo tratamos como caja negra. Lo que sabemos es su salida continua, una probabilidad de fraude entre 0 y 1, en la columna `big_model_fraud_probability`. Nuestro objetivo es entrenar un **TinyModel** — un perceptrón simple, no un MLP — que replique esa salida.

Hay un detalle clave: el dataset también tiene una columna `flagged_fraud` con la etiqueta binaria de fraude real. *El enunciado prohíbe usarla en el entrenamiento*. La usamos sólo para evaluación con métricas de clasificación (precision, recall, F1).”

[Slide 8, features y código] “Nueve features: `amount_usd`, `quantity_purchased`, `session_duration_sec` etc. Excluimos tres por irrelevantes o no numéricas: `timestamp`, `device_screen_resolution`, `time_since_last_login_s`. Pueden ver la lista en `ejercicio1/nonlinear_perceptron/configs/baseline.json`.

El código vive en `ejercicio1/nonlinear_perceptron/nonlinear_perceptron.py`: K-fold estratificado por `flagged_fraud`, normalización z-score *fit-on-train-only* para evitar leakage, y online learning sample-by-sample.”

## 2.2 Lineal vs No-Lineal (1:00 – 2:30)

[Slide 9, dos versiones] “Probamos dos versiones del perceptrón:”

- **Lineal (Adaline)**: salida  $O = w \cdot x$ , regla delta  $\Delta w = \eta (z - O) x$ .
- **No-Lineal (sigmoide)**: salida  $O = \sigma(w \cdot x) = 1/(1 + e^{-w \cdot x})$ , regla delta con derivada de la sigmoide:  $\Delta w = \eta (z - O) O(1 - O) x$ .

“El factor  $O(1 - O)$  modula el gradiente: cuando  $O$  está cerca de 0 o 1 (zonas saturadas), el gradiente se achica. Esa es justamente la diferencia que pide *el apunte de perceptrón*.”

[Slide 10, MSE comparison] “En distillation pura, MSE contra `big_model_fraud_probability`, K-fold=5 estratificado:

| Modelo    | MSE test (mean $\pm$ std)             |
|-----------|---------------------------------------|
| Lineal    | 0,0266 $\pm$ 0,0008                   |
| No-Lineal | <b>0,0110 <math>\pm</math> 0,0004</b> |

El no-lineal reduce el MSE en **59 %**. La razón es directa: el target está acotado en  $[0, 1]$ , la sigmoide mapea naturalmente ahí, el lineal puede producir valores fuera del rango y el MSE lo penaliza.”

## 2.3 Threshold sweep (2:30 – 4:00)

[Slide 11, el problema del threshold default] “Acá hay una sutileza importante. Con threshold default 0,5, el no-lineal da **precision = 0.33 con recall = 1.0**. Es decir, clasifica casi todo como fraude. Eso parece terrible, pero no es el modelo: es el threshold.

Las salidas sigmoide se concentran: la mediana de los scores de fraude está en 0.99. O sea, los fraudes están muy arriba, pero el modelo aprendió a no usar el rango bajo de la sigmoide para los no-fraudes — los pone en valores como 0.4 o 0.6, no en 0.05.”

[Slide 12, threshold óptimo] “Hicimos un sweep de threshold de 0.00 a 1.00 con paso 0.01. El óptimo por F1 es **threshold = 0.89**: precision 0.886, recall 0.861, F1 0.873, accuracy 97.1 %. AUC-ROC del modelo: **0.9921**. El lineal está en 0.9720.

Si el cliente prioriza no auditar falsos positivos,  $\text{threshold} \geq 0.95$  da  $\text{precision} \geq 0.95$  con  $\text{recall} 0.75$  — pierde sensibilidad pero gana precisión. La elección depende del costo asimétrico entre falso positivo y falso negativo del negocio.”

**Transición (4:00 – 5:00):** “Eso cierra el ejercicio 1. La diferencia con los siguientes dos ejercicios es que acá tenemos un perceptrón simple — una sola capa. Para clasificar dígitos manuscritos en 10 clases necesitamos un MLP. Le paso a [P3].”

### 3. Persona 3 — Ejercicio 2 (Clasificación de dígitos)

**Objetivo:** explicar el workflow disciplinado Fase 1 → Fase 2 → final\_eval, mostrar los sweeps con sus ganadores, y entregar el resultado del final\_eval con el cachetazo del distribution shift como gancho para Persona 4.

#### 3.1 Datos y problema (0:00 – 0:40)

[Slide 13, problema] “Clasificación multiclase de dígitos del 0 al 9. Cada muestra es un *vector de 784 floats en [0,1] que representa una imagen de 28 por 28*. Train: `digits.csv` con 12.449 muestras. Test (producción): `digits_test.csv` con 2.498 muestras.

Una regla que el enunciado deja implícita pero que adoptamos como ley: `digits_test.csv` se evalúa una sola vez, al final. No se toca durante búsqueda de hiperparámetros.”

#### 3.2 Workflow Fase 1 + Fase 2 (0:40 – 2:00)

[Slide 14, workflow disciplinado] “El espacio de hiperparámetros es grande: arquitectura, optimizador, learning rate, batch size —  $O(n^4)$ . Para no caer en sweep aleatorio dividimos en dos fases:

**Fase 1 (`k_folds=1`, exploratoria).** Sweeps secuenciales: primero arquitectura, después optimizador, después learning rate, después batch size. Cada sub-fase usa el ganador del anterior. Eso reduce  $O(n^4)$  a  $O(4n)$ .

**Fase 2 (`k_folds=5`, validación).** Una vez consolidado el `base.json`, variamos un solo HP a la vez y volvemos a evaluar con K-fold completo. Da comparativas limpias para esta presentación.”

[Slide 15, ganadores Fase 1] “Resultados de las 16 corridas de Fase 1:

| Sub-fase      | Ganador            | val_acc | Comentario                                     |
|---------------|--------------------|---------|------------------------------------------------|
| Arquitectura  | [784, 100, 50, 10] | 0.9674  | elegida por parsimonia (vs [784, 128, 64, 10]) |
| Optimizador   | adam, lr=0.001     | 0.9674  | SGD no convergió en 50 epochs                  |
| Learning rate | 0.001              | 0.9674  | 0.0001 lento, $\geq 0,005$ overshoot           |
| Batch size    | 16                 | 0.9674  | best_ep=3, el más rápido                       |

El `base.json` consolidado, corrido con K=5, da **val\_acc 96.22 %  $\pm$  0.43 %**.”

#### 3.3 Fase 2 valida la elección (2:00 – 3:30)

[Slide 16, plots Fase 2] “Fase 2 es one-at-a-time sobre el base. Las cuatro dimensiones que barriremos: LR, arquitectura, optimizador, inicializador. Pueden ver los plots en `ejercicio2/presentacion/`.

- LR: el único sweep con rangos catastróficos. lr=10 deja la red en accuracy random (12 %); lr=0.1 también diverge. lr=0.001 sigue siendo el ganador.
- Arquitectura: `wider` gana por 0.17pp pero tarda 3.6 veces más → empate técnico, mantenemos base.
- Optimizador: momentum y adam empatan dentro del std.
- Inicializador: he, xavier, uniform los tres en empate técnico.

Lo importante: **Fase 2 valida la elección de Fase 1 más que descubrir un ganador nuevo.** Eso era el objetivo — la metodología coarse-to-fine ya hizo el trabajo.”

### 3.4 Final eval — el descubrimiento (3:30 – 5:00)

[Slide 17, curvas + matriz base] “Antes del final\_eval, el modelo se ve sano. Curvas suaves, convergencia en 5 epochs, matriz de confusión con diagonal limpia para 9 de 10 clases. La excepción: la clase 8 colapsa a precision 0 y recall 0 en val. Indicador de algo raro en la distribución del train.”

[Slide 18, final\_eval Ej2] “Final eval: entrenamos con todo `digits.csv` y evaluamos UNA SOLA VEZ contra `digits_test.csv`. Resultado:

val K-fold=5: **96.22 %** → test: **86.30 %**

**Drop de 10 puntos porcentuales.** Eso fue una sorpresa fuerte. La matriz de confusión sobre test confirma que las confusiones se reparten por todo el grid, no son una clase específica fallando.”

[Slide 19, hipótesis] “La interpretación: **el modelo está bien, los datos no se parecen.** *El enunciado del ejercicio 3 menciona que más datos pueden ayudar* — y eso fue justamente la hipótesis que queríamos testear.”

**Transición (5:00):** “Le paso a [P4] que cuenta cómo intentamos cerrar ese gap.”

## 4. Persona 4 — Ejercicio 3 (Pack C, búsqueda del 98 %)

**Objetivo:** contar la iteración Pack C, el descubrimiento del +10pp por más datos, el techo del 96.88 %, y por qué no llegamos al 98 %. Cerrar con conclusiones del TP.

### 4.1 Hipótesis y plan (0:00 – 0:40)

[Slide 20, plan] “Si el problema es distribution shift y no falta de capacidad, el primer movimiento natural es **más datos**. *El enunciado del ejercicio 3 nos provee `more_digits.csv` con 15.742 muestras adicionales.* Si eso no alcanza, activamos Pack C: regularización L2, dropout, augmentation gaussiana, lr\_schedule. Si nada llega al 98 %, escalamos capacidad.

El engine `mlp/` ya soporta esto: el config tiene un campo `regularization` y `extra_csv_paths` concatena CSVs antes del split. Cero cambios al training loop.”

### 4.2 El primer movimiento dominó (0:40 – 1:40)

[Slide 21, more\_digits] “Concatenamos `more_digits.csv` sin tocar nada más. Resultado:

val K=5: 96,22 % → 97,06 %    y    test: **86,30 %** → **96,36 %**

**+10 puntos porcentuales en test, cero cambios al modelo.** Confirma la hipótesis. El macro F1 saltó de 0.857 a 0.959 → la clase 8 que estaba colapsada en Ejercicio 2 ahora se predice correctamente.”

### 4.3 Iteración Pack C (1:40 – 3:00)

[Slide 22, tabla Pack C] “Sobre eso aplicamos las técnicas Pack C, primero individuales, después combinaciones. Doce configs evaluadas en total — van a ver la tabla completa.

| Config             | val K=5       | test          | $\Delta$     |
|--------------------|---------------|---------------|--------------|
| base_extra_data    | 0.9706        | 0.9636        | — (baseline) |
| + L2               | 0.9758        | 0.9656        | +0,20        |
| + dropout          | 0.9750        | 0.9628        | −0,08        |
| + L2 + dropout     | 0.9772        | 0.9604        | −0,32        |
| + <b>L2 + aug</b>  | <b>0.9760</b> | <b>0.9688</b> | <b>+0,52</b> |
| + wider + L2 + aug | 0.9777        | 0.9640        | +0,04        |

**Ganador: L2 (1e-4) + augmentation gaussiana ( $\sigma = 0,05$ )  $\rightarrow$  96.88 % en test.** Eso requirió **implementar dropout, augmentation y lr\_schedule en el engine** — los detalles de implementación están en `mlp/network.py`, con tests verde.”

#### 4.4 Análisis: qué movió la aguja (3:00 – 4:00)

[Slide 23, qué ayudó / qué no] “El análisis honesto:

**Lo que ayudó:**

- Más datos: +10pp. El movimiento dominante.
- L2: +0.20pp consistente.
- Aug gaussiana: +0.32pp *adicional* sobre L2 (sinergia con L2).

**Lo que no ayudó:**

- Dropout: ayuda en val, no transfiere a test. Reduce varianza específica al fold, no al shift.
- **wider** [784, 200, 100, 10]: mejor en val, peor en test. Más capacidad memoriza el train, no generaliza. Eso descarta que el problema sea capacidad.
- Combinar todo a la vez: la regularización compite consigo misma. L2+dropout+aug fue peor que L2+aug solo.

”

#### 4.5 Por qué no llegamos al 98 % (4:00 – 4:30)

[Slide 24, hipótesis del techo] “El target del 98 % no lo alcanzamos. Mejor: 96.88 %, faltan 1.12pp. Tres hipótesis:

1. **Augmentation insuficiente:** gaussian noise es isotrópico — ruido independiente por píxel. El shift entre train y test parece ser de *estilo de escritura* (rotación, traslación, grosores). Eso pide augmentation por transformaciones geométricas, no por noise. No la implementamos — gaussian noise era la primera aproximación.
2. **Sub-representación:** el `final_eval` usa val 10 % random para early stopping. Si los digits “difíciles” del test no aparecen ahí, la red para antes de tiempo.
3. **Capacidad:** subjetivamente improbable porque **wider** sobreajustó. Pero quizás con augmentation geométrica una arquitectura mayor sí ayudaría.

”

#### 4.6 Conclusiones del TP (4:30 – 5:00)

[Slide 25, conclusiones] “Tres ideas para llevarse:

1. Una buena pipeline gana más que tunear hiperparámetros: el +10pp del Ejercicio 3 vino de *datos*, no de modelo.

2. Los empates técnicos (diferencia menor a 0.3pp con  $\text{std} \approx 0,4\%$ ) son mucho más comunes que los wins claros. Hay que elegir por parsimonia, no por 0.05pp en val.
3. “Mejor en val” no es “mejor en test” cuando hay shift. Dropout fue el ejemplo: ganador en val K-fold, perdedor en test.

”

[Slide 26, cumplimiento + cierre] “Cumplimos todo lo de la consigna excepto el 98 % del Ejercicio 3. Está documentado honestamente con hipótesis verificables en `ejercicio3/README.md`. El informe técnico completo en `docs/informe_tp3.pdf`, la guía operativa en `docs/guia_compañeros.md`, y el repo en `github.com/pi-na/SIA-TP3` branch `tp3-mlp`. Gracias. Preguntas.”

## Apéndice — preguntas anticipadas y cómo responder

¿Por qué no usaron sklearn / PyTorch? *El enunciado lo prohíbe explícitamente.* La consigna pide perceptrones desde cero con NumPy. Es parte de la evaluación.

¿Por qué dos fases en el Ejercicio 2? Para no caer en sweep aleatorio. Fase 1 con `k_folds=1` explora rápido y reduce  $O(n^4)$  a  $O(4n)$ ; Fase 2 con `K=5` valida. Sin Fase 2, una elección que ganó por suerte en un solo fold parece definitiva.

¿Por qué no llegan al 98 % del Ejercicio 3? (Si la pregunta es hostil) “Identificamos que el techo viene de la calidad del augmentation, no de capacidad. Probamos las cuatro técnicas Pack C del enunciado y combinaciones. Lo que faltaba implementar era augmentation por transformaciones geométricas — queda explicitado como follow-up. La autocritica está en el README.”

¿Por qué el Ejercicio 1 está aparte del engine? Lo implementé hace un mes con scripts independientes, antes de unificar la codebase. Funcionalmente es correcto y los resultados son válidos; mantener la implementación separada preserva la trazabilidad histórica del repo.

¿Cómo se reproducen los resultados? Cualquiera puede correr `python3 -m mlp.train -config <ruta> -csv-root . -output-dir <out>`. El ganador del Ej3 es `ejercicio3/configs/pack_c/12_aug.json`. Comandos exactos están en el apéndice del informe técnico.

¿Por qué eligieron `threshold=0.89` en el Ejercicio 1? (P2) Porque maximiza F1 en el sweep. Si el cliente prefiere precisión sobre recall, `threshold=0.95` o más alto. Es una decisión de negocio sobre el costo asimétrico entre falso positivo y falso negativo.

¿Qué bugs encontraron? (P1, si hay tiempo) Cuatro reales durante el desarrollo: predicción bipolar mal codificada, identity activation aliasaba su input, identity row spread perdía strings en el `run_summary`, `numerical_grad` con int dtype daba garbage. Todos detectados por los tests del XOR y AND *antes* de tirar el código a dígitos.